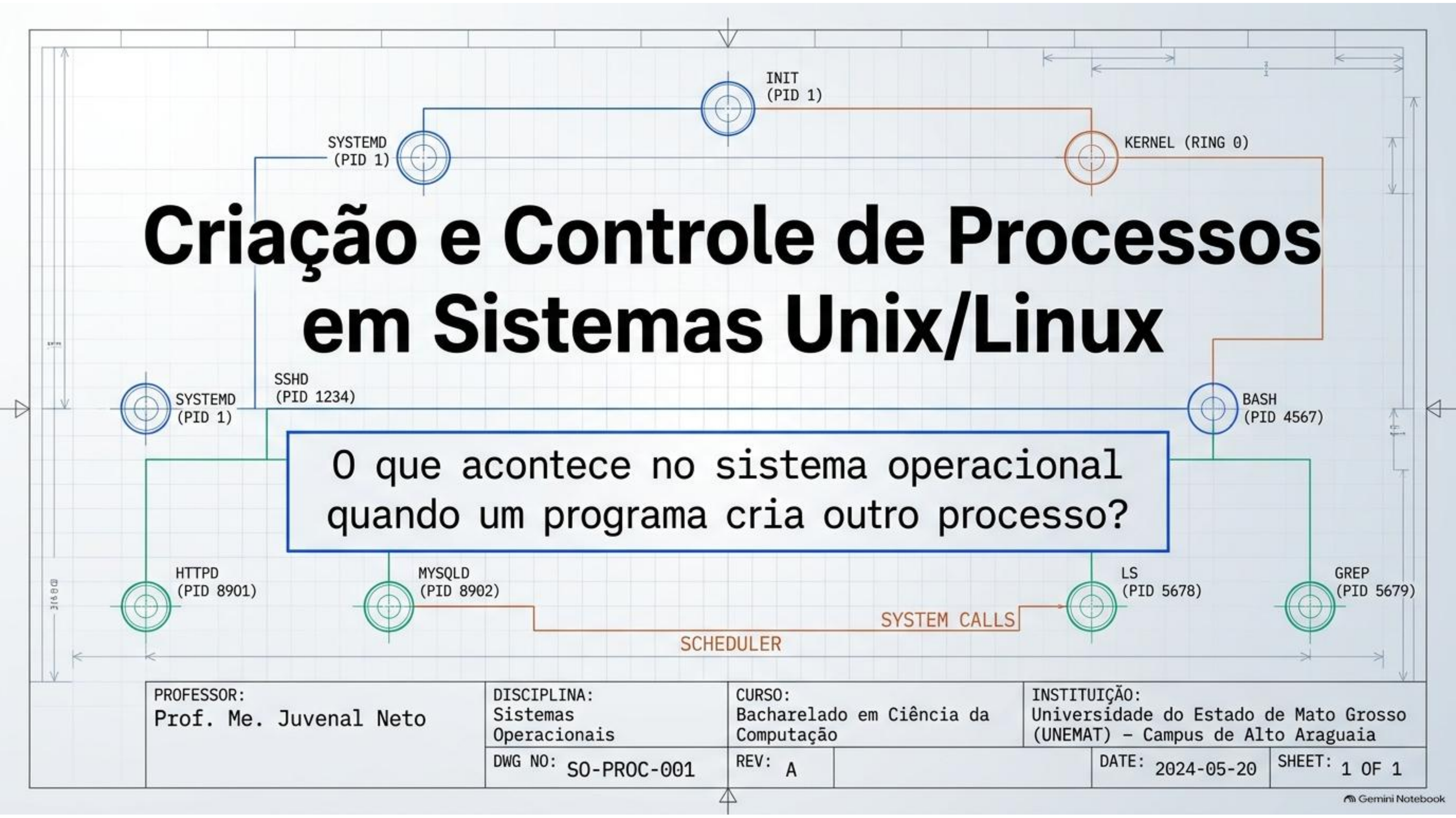


Criação e Controle de Processos em Sistemas Unix/Linux

O que acontece no sistema operacional quando um programa cria outro processo?

PROFESSOR: Prof. Me. Juvenal Neto	DISCIPLINA: Sistemas Operacionais	CURSO: Bacharelado em Ciência da Computação	INSTITUIÇÃO: Universidade do Estado de Mato Grosso (UNEMAT) – Campus de Alto Araguaia
	DWG NO: SO-PROC-001	REV: A	DATE: 2024-05-20 / SHEET: 1 OF 1

Gemini Notebook

O QUE É UM PROCESSO?

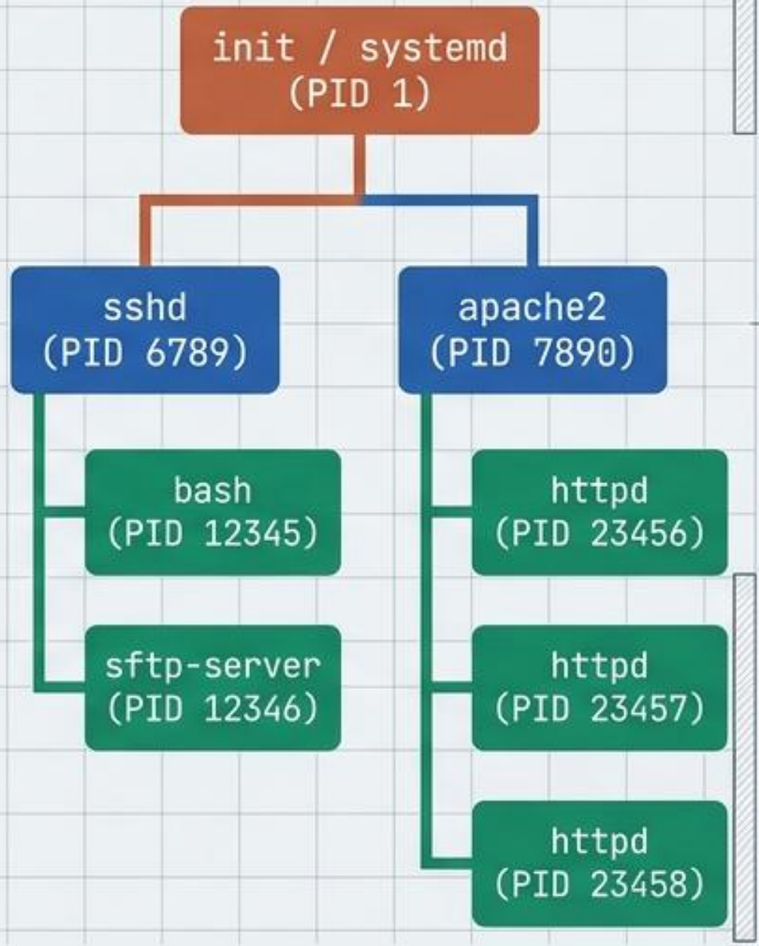
Programa em Execução (Entidade Ativa).



O PCB (PROCESS CONTROL BLOCK)



A ÁRVORE GENEALÓGICA



Gemini Notebook

fork() duplica o programa ou cria um processo?
Resposta: Cria um processo duplicando todo o contexto do chamador!

Antes

Processo Pai (PID 100)

Código
0111011100 0110011110
0111101111 0111101011

Dados
var1 = 10 ptr = 0x...
var = 0x... ptr = 0x...
var = 0x... ...

PCB
PID: 100
PPID: [Parent PID]
Estado: EXECUTANDO
PC (Program Counter): [...]
Registros: ...
Ponteiros de Memória: ...

Técnica Copy-on-Write: Compartilha a memória física até que um tente gravar.

```c
#include <stdio.h>
#include <unistd.h>

int main() {
  printf("Antes do fork\n");

  pid_t pid = fork(); // Ponto de divisão

  printf("Após o fork, PID: %d\n", getpid());
  // Ambos os processos continuam a partir daqui
  return 0;
}
```

Depois

Processo Pai (PID 100)

Código
0111011000 011001110
0111101111 011110011

Dados
var1 = 10 ptr = 0x...
var = 0x... ptr = 0x...
var = 0x... ...

PCB
PID: 100
PPID: [Parent PID]
Estado: EXECUTANDO
PC (Program Counter): [...]
Registros: ...
Ponteiros de Memória: ...

Processo Filho (PID 101)

Código
0111011000 011001110
0111101111 011111011

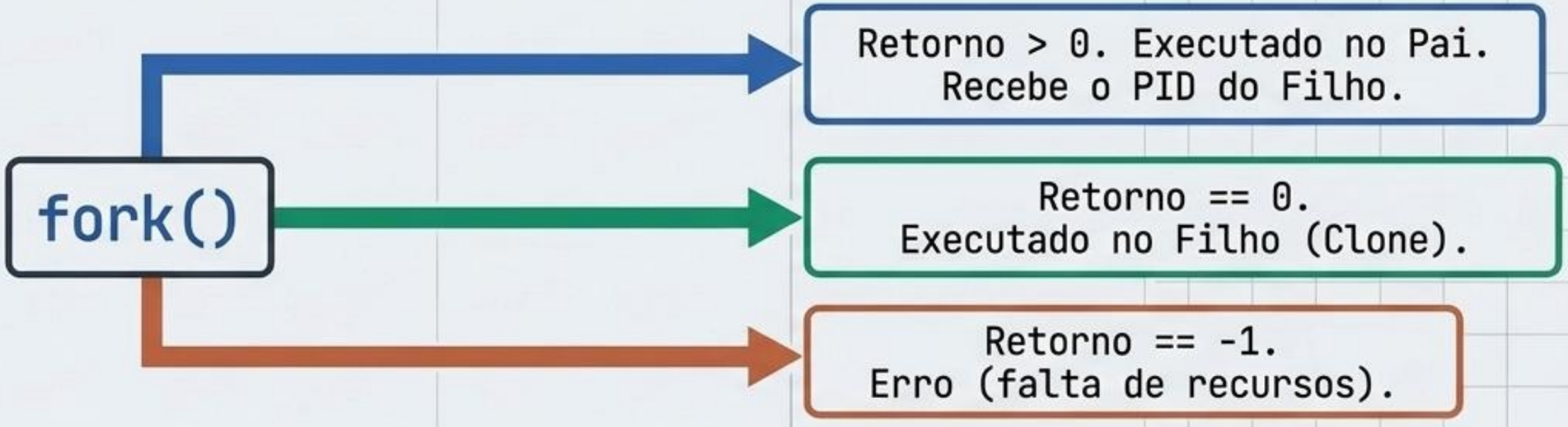
Dados
var1 = 10 ptr = 0x...
var = 0x... ptr = 0x...
var = 0x... ...

PCB
PID: 101
PPID: 100
Estado: EXECUTANDO
PC (Program Counter): [...]
Registros: ...
Ponteiros de Memória: ...

Ambos passam a executar a partir da instrução exatamente seguinte.

Gemini Notebook

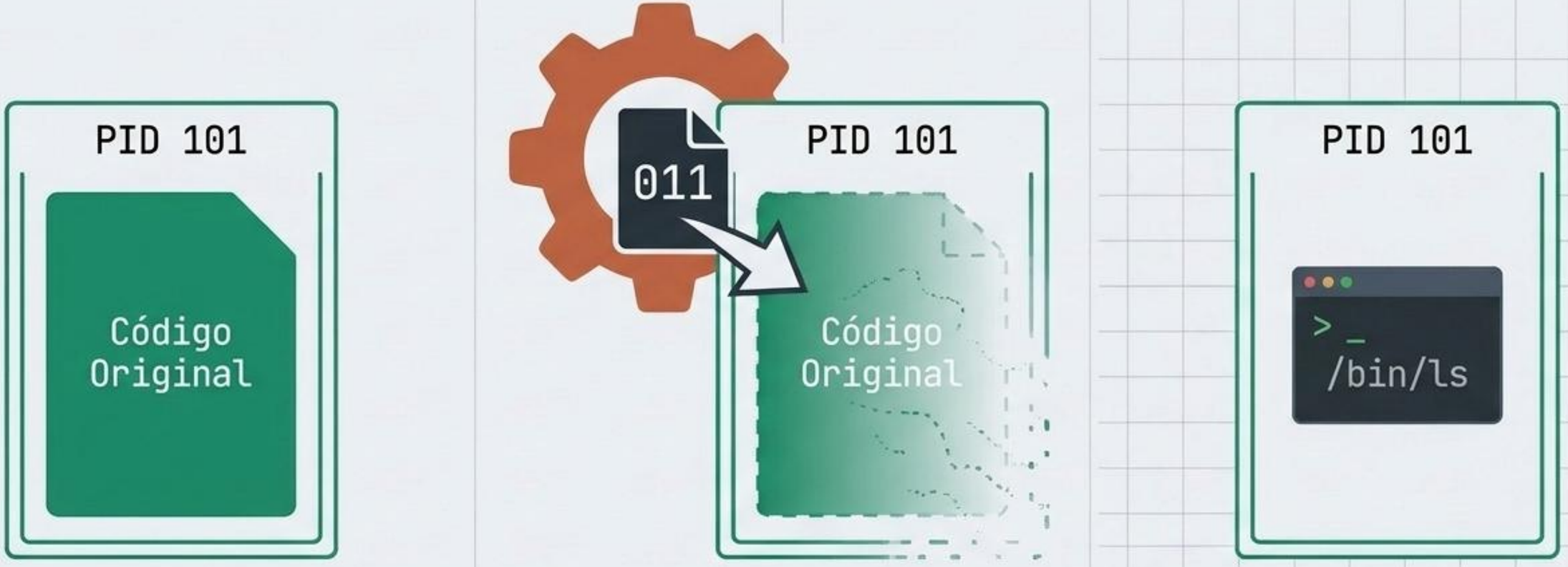
Quem executa a próxima instrução após o fork?
Ambos, mas recebem valores diferentes.




```
if (pid == 0) {
    // Código do Filho
} else if (pid > 0) {
    // Código do Pai
}
```

Gemini Notebook

exec() cria um novo PID? NÃO!

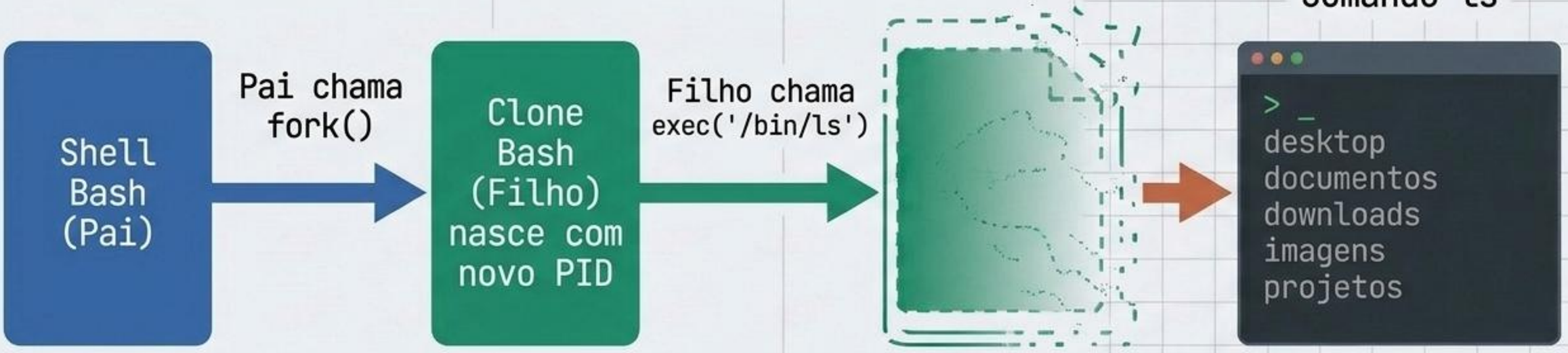


A família exec() substitui a imagem de memória atual por um novo programa binário. O processo é reciclado, mantendo o mesmo PID. Se houver sucesso, exec() não retorna.

Gemini Notebook

O Padrão de Ouro: fork() + exec()

Como um terminal (Shell) executa seus comandos.

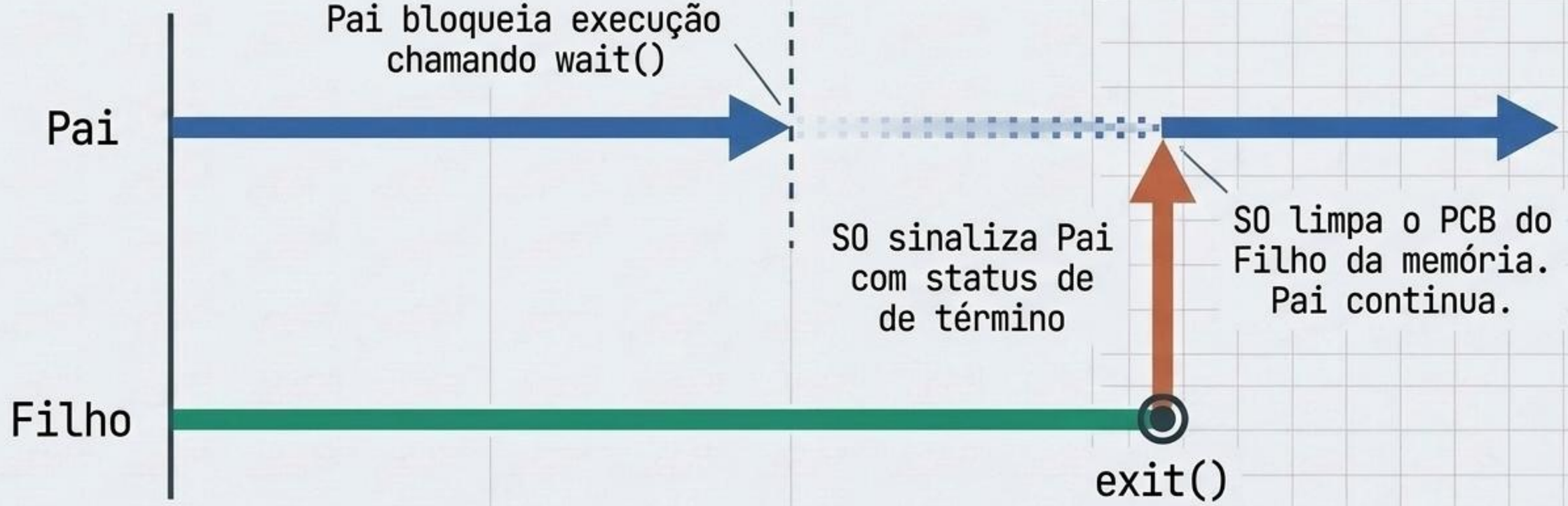


Filho sofre mutação, lista arquivos e termina.

Gemini Notebook

A Sincronização Paterna: A Chamada wait()

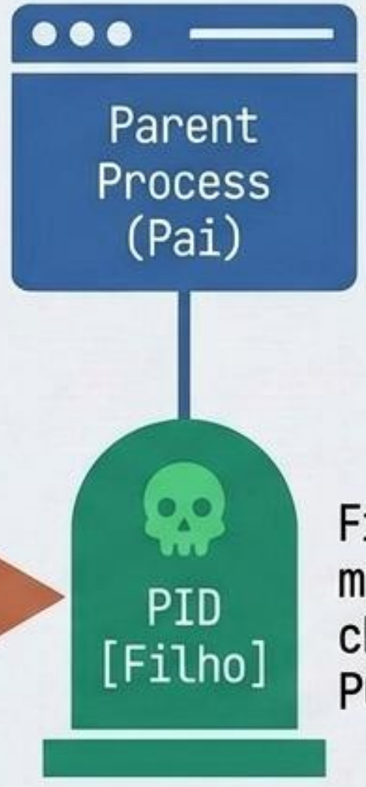
Por que existe wait()?



Gemini Notebook

Anomalia no Ciclo 1: Processos Zombie

Um zombie ainda está executando e consumindo CPU? Não!



exit()

Filho terminou (exit), mas Pai ainda não chamou wait(). PCB preso na tabela.

```
> _

UID        PID  PPID STAT CMD
user      1000   900 S    bash
user      1001  1000 Z    [programa] <defunct>
```

Gemini Notebook

Anomalia no Ciclo 2: Processos Órfãos (*Orphans*)

Passo 1: Estado Inicial

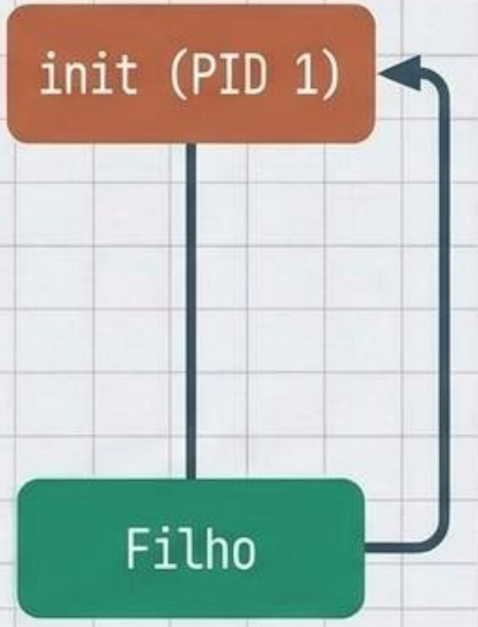


Passo 2: Terminação do Pai



Pai termina antes do Filho.

Passo 3: Adoção pelo init



Adoção: O processo raiz (PID 1) adota o órfão e fará o wait() no futuro.

Gemini Notebook

Matriz Diagnóstica: Zombie vs. Órfão

Zombie e Órfão são a mesma coisa? Não, são dinâmicas temporalmente opostas.

Condição	Quem terminou primeiro?	Impacto no SO	Tratamento
Zombie	Filho	Ocupa slot (PCB) na tabela de processos indevidamente.	Pai precisa executar wait() ou o Pai precisa ser morto.
Órfão (*Orphan*)	Pai	Continua executando em background, solto.	Adoção automática pelo init (PID 1), que providenciará a limpeza.

Gemini Notebook

Linux Lab 1: A Relação Pai & Filho

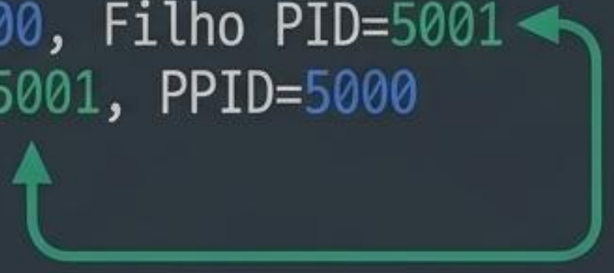
Observar a genealogia e os PIDs concorrentes.

Código Fonte (C)

```
pid = fork();
if (pid == 0) {
   sleep(2);
   printf("Filho: PID=%d, PPID=%d",
       getpid(), getppid());
} else {
   printf("Pai: PID=%d, Filho PID=%d"
     ", getpid(), pid);
   wait(NULL);
}
```

Terminal Output (Simulação)

```
$ gcc lab1.c -o lab1
$ ./lab1
Pai: PID=5000, Filho PID=5001
Filho: PID=5001, PPID=5000
```



Gemini Notebook

Linux Lab 2: Criando e Caçando Zombies

Código Fonte (C)

Falha Intencional

```
pid = fork();
if (pid == 0) {
   exit(0); // Filho morre imediatamente
} else {
   sleep(15); // Pai dorme, NÃO chama
   wait()
}
```

Terminal Output (Simulação)

```
$ ps -o pid,ppid,state,cmd | grep Z
  PID  PPID S CMD
 6001  6000 Z [lab2] <defunct>
```

Gemini Notebook

Linux Lab 3: Órfãos e Transmutações (exec)

Cenário A: Órfão

Pai faz fork() e exit(0). Filho dorme e depois checa PPID.

Terminal Output (Simulação)

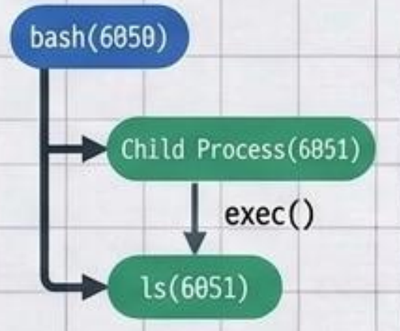
```
$ ./lab3a
Pai: PID=6000, PPID=5000
Pai: exit(0)
Filho: PID=6001, PPID=6000, dormindo...
Filho: acordando...
Filho: PID=6001, PPID=1 (init/systemd)
```

Cenário B: Transmutação com exec

Filho executa execlp('ls', 'ls', NULL).

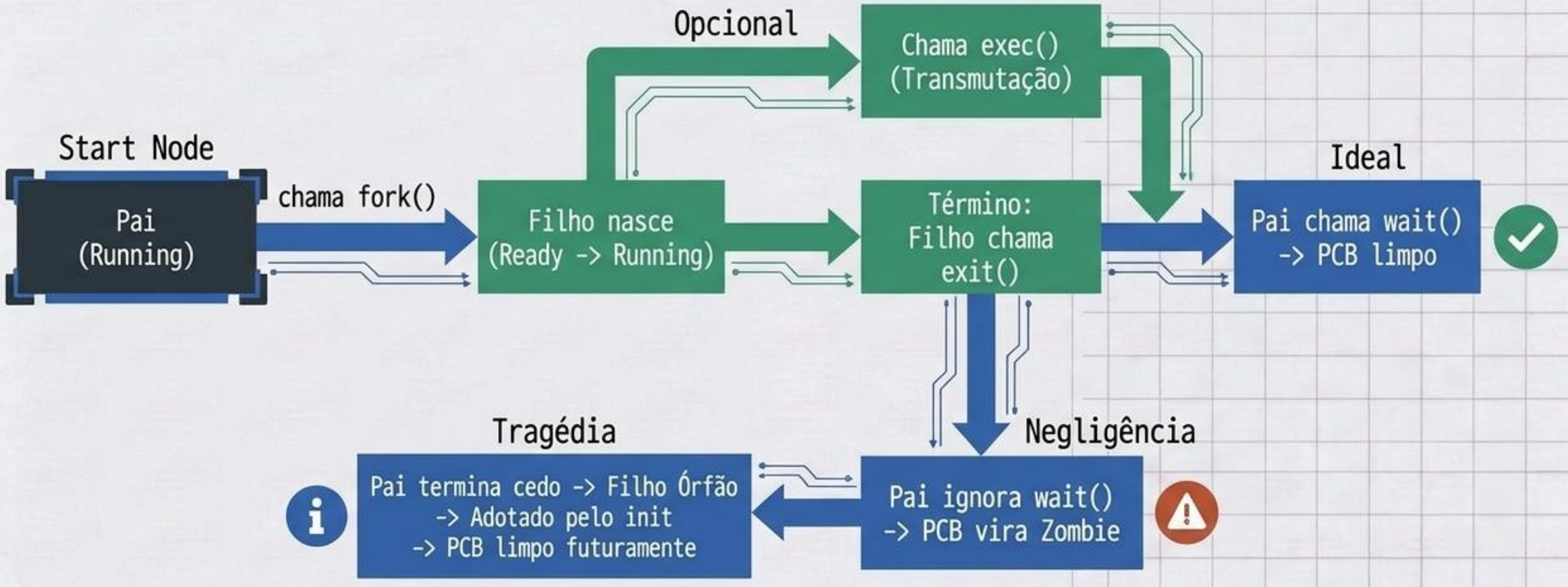
Terminal Output (Simulação)

```
$ ./lab3b
Pai: PID=6050, PPID=5000
Filho: PID=6051, executando execlp...
$ pstree -p 6050
bash(6050)──ls(6051)
```



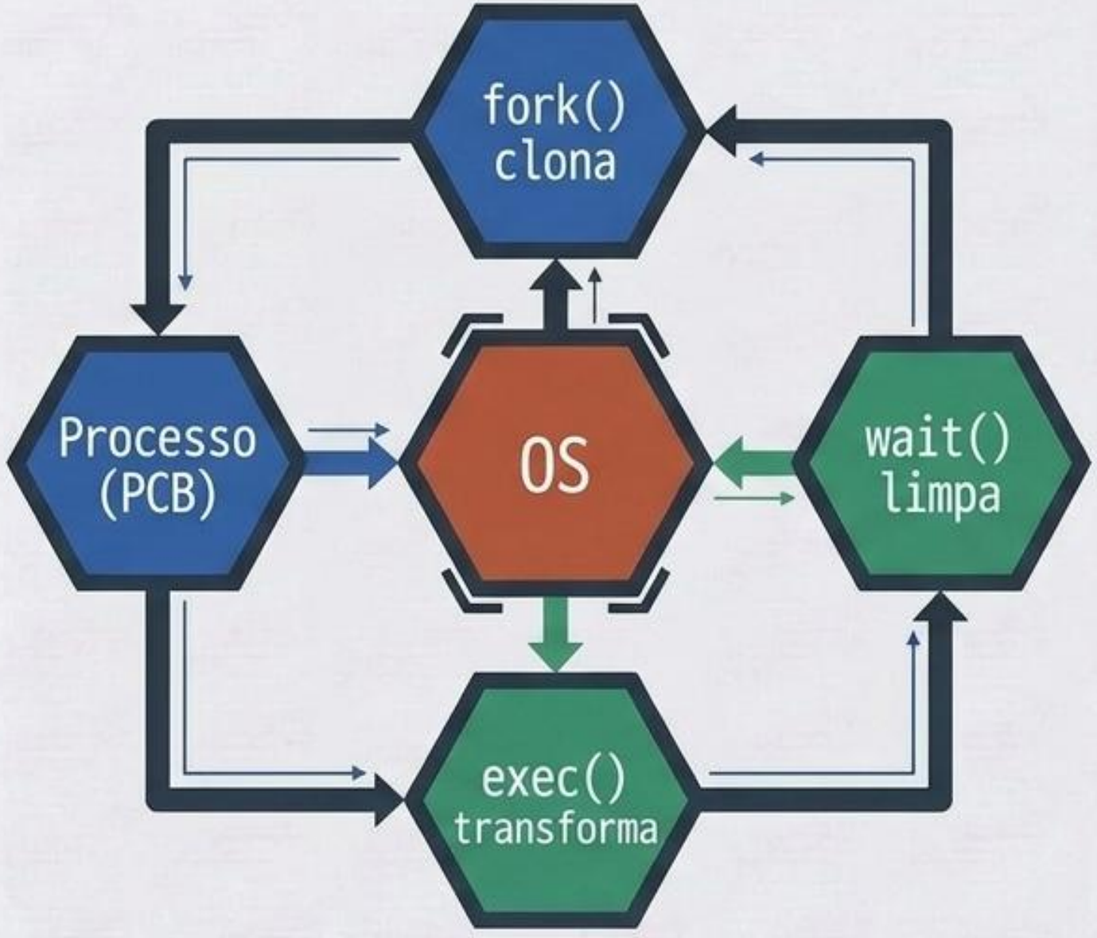
Gemini Notebook

O Blueprint: O Ciclo de Vida Completo do Processo



Gemini Notebook

Síntese do Sistema: O que levaremos desta aula?



Processos são cópias: Nascem de ancestrais via `fork()` e são moldados por novas identidades via `exec()`.

Processos exigem limpeza:
Requerem coordenação estrita do Pai (`wait()`) para limpar a tabela do kernel, senão tornam-se Zombies.

A base do terminal:
A teoria destas chamadas de sistema é a engenharia invisível que faz o seu shell Linux funcionar a cada ENTER.

Gemini Notebook